CS4475 Fall 2026 Lab Report
Gael Sustaita Submitted on 10/14/2026
gsustaita4@miners.utep.edu

PA2: Priority-based Scheduler for xv6

Repository: https://github.com/gael68/xv6-riscv, branch `pa2`. As the handout suggests, `pa2` starts from my `pa1-syscalls` branch, so the PA1 `time`, `matmul`, `sleep` and `wait2`/`cputime` work is available. Upstream base is mit-pdos/xv6-riscv commit `9e3161a`. Built with `riscv64-elf-gcc` and run on `qemu-system-riscv64` (macOS, Apple Silicon). As required, `CPUS := 1` is set in the Makefile. One xv6 tick is one timer interrupt, about 0.1 s.

Build configuration: the default build (what is committed and what all `usertests` ran on) is `SCHEDPOLICY = SCHED_PRIO` with `AGING = 1`, set in `kernel/param.h`. For comparison runs I built `make clean; make qemu SCHEDPOLICY=SCHED_RR` (round robin) and `make clean; make qemu AGING=0` (priority without aging). Each task is its own commit on `pa2`, so `git log` shows the code at every step. This report is committed as `PA2 Report.pdf` at the top of the repository.

Priority convention: a larger number means a higher priority, as in the handout's sample output, where the priority-5 `matmul` receives most of the CPU. Base priorities are 0–49 (`MAXPRIORITY`); effective priorities with aging are 0–99 (`MAXEFFPRIORITY`). `init` starts at 0 and children inherit their parent's priority.

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

Files changed.

File	Change	Purpose
`kernel/pstat.h`	Added `struct pstat` (pid, state, size, ppid, name). Moved `enum procstate` here from `proc.h`.	Shared by the kernel and `ps`. User programs cannot include `proc.h` (it needs spinlocks, contexts, ...), but `ps` needs the state names.
`kernel/proc.h`	`#include "pstat.h"` in place of the enum.	Every kernel file that uses `struct proc` still sees `enum procstate`.
`kernel/proc.c`	New `kgetprocs(uint64 addr)` (line 505).	Walks the process table and copies one `struct pstat` per in-use process to user memory.
`kernel/sysproc.c`	New `sys_getprocs()` (line 49).	Fetches the user pointer with `argaddr()` and calls `kgetprocs()`.
`kernel/syscall.h`, `kernel/syscall.c`	`SYS_getprocs 24` and its table entry.	Dispatches system call 24.
`kernel/defs.h`	Prototype for `kgetprocs`.	

File	Change	Purpose
`user/usys.pl`, `user/user.h`	`entry("getprocs");int getprocs(struct pstat*);`	User-space stub and prototype, as the handout says.
`user/ps.c`, `Makefile`	Provided `ps.c` copied in; `_ps` added to `UPROGS`.	The test program.

How it works.

- `kgetprocs()` loops over `proc[NPROC]`. For each slot it takes `wait_lock` and then `p->lock` (the order xv6 uses elsewhere). `p->lock` protects `state`, and `wait_lock` keeps `p->parent` from being freed while its pid is read. `UNUSED` slots are skipped.
- The fields are copied into a `struct pstat` on the kernel stack. The locks are released before `copyout()`, which may need to fault in a lazily allocated user page. Entry n goes to `addr + n * sizeof(struct pstat)`.
- The return value is the number of entries copied. If `copyout()` fails (bad user pointer) the call returns -1. Task 2's `prioritytest` checks this case.
- **Problem with the provided ps.c:** it did not compile. Current xv6 `printf` has no `%l`, and with `-Werror` the build stops at `unknown conversion type character`. I changed `%l` to `%ld` for the `uint64` size field.

```
// Copy a struct pstat for each process in use to the user
// array at addr, which must have room for NPROC entries.
// Return the number of processes, or -1 on error.
int
kgetprocs(uint64 addr)
{
  struct proc *p;
  struct proc *me = myproc();
  struct pstat ps;
  int n = 0;

  for (p = proc; p < &proc[NPROC]; p++) {
    // wait_lock keeps p->parent from being freed while we read its pid.
    acquire(&wait_lock);
    acquire(&p->lock);
    if (p->state == UNUSED) {
      release(&p->lock);
      release(&wait_lock);
      continue;
    }
    ps.pid = p->pid;
    ps.state = p->state;
    ps.size = p->sz;
    ps.ppid = p->parent ? p->parent->pid : 0;
    safestrcpy(ps.name, p->name, sizeof(ps.name));
    release(&p->lock);
    release(&wait_lock);

    // copyout() may fault in a lazily allocated page, so don't hold locks.
    if (copyout(me->pagetable, me->sz, addr + n * sizeof(ps), (char *)&ps,
                sizeof(ps)) < 0)
      return -1;
    n++;
  }
  return n;
}
```

Results. Background jobs are on their own command lines so that `ps` runs after they start:

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
3       running         16384   2       ps
$ sleep 50 &
$ matmul 10 &
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
8       running         16384   2       ps
5       runnable        16384   1       sleep
7       runnable        16384   1       matmul
$ echo waiting
waiting
$ Time: 60 ticks
ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
10      running         16384   2       ps
$
```

`ps` lists itself as running and `init`/`sh` as sleeping. The background `sleep` and `matmul` have ppid 1: the shell runs a background job in a child shell that exits right away, so the job is reparented to `init`, as in the handout's sample. `sleep` appears as runnable rather than sleeping because `pause()` sleeps on `&ticks`, and the clock interrupt wakes it every tick to check whether its time is up. After `matmul` finishes it no longer appears.

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

Files changed.

File	Change	Purpose
`kernel/proc.h`	`int priority;` in `struct proc`.	Base priority, 0 (lowest) to 49.
`kernel/proc.c`	`allocproc()`: `p->priority = 0` (line 128). `kfork()`: `np->priority = p->priority` (line 294).	`init` (created by `allocproc()` in `userinit()`) starts at 0. Children inherit the parent's priority.
`kernel/param.h`	`#define MAXPRIORITY 49`	One place for the limit; used by the kernel, `pexec`, `sh` and the tests.
`kernel/sysproc.c`	`sys_getpriority()` (line 57), `sys_setpriority()` (line 63).	`setpriority(n)` returns -1 unless 0 ≤ n ≤ 49, otherwise sets the field under `p->lock` and returns 0.
`kernel/syscall.[ch]`, `user/usys.pl`, `user/user.h`	System calls 25 and 26 and their stubs/prototypes.	
`kernel/pstat.h`, `kernel/proc.c`	`int priority` added to `struct pstat`; `kgetprocs()` copies it.	The change to `getprocs()` asked for in this task.

File	Change	Purpose
`user/ps.c`	New `prio` column; `USED` added to the state names; error message if `getprocs()` fails.	A process being created by `fork()` is briefly `USED`; without the entry `ps` would print a null string for it.
`user/pexec.c`	Provided program, with input checking added.	See below.
`user/prioritytest.c`	New test program.	Checks the system calls, inheritance, and error cases.

Changes to pexec. The provided version called `atoi()` on the priority (so `abc` silently became 0), ignored `setpriority()`'s result, copied arguments into a fixed 16-entry array without a bounds check, and did not handle `fork()`/`exec()` failures. My version rejects a priority that is not a number from 0 to 49, passes `argv + 2` straight to `exec()` (it is already null-terminated, so no copy and no overflow), prints an error if `fork`, `exec` or `wait` fails, and reports a non-zero exit status of the command.

prioritytest sets and reads priorities, including the out-of-range values -1 and 50, which must fail and leave the priority unchanged. It calls `getprocs()` with an invalid pointer, which must return -1. It then forks a child that reports its inherited priority through a pipe and changes its own priority to 10, and it runs `ps` so both processes are visible.

Results.

```
xv6 kernel is booting


init: starting sh
$ prioritytest
initial priority: 0
setpriority(25): 0 (expected 0) ok
getpriority(): 25 (expected 25) ok
setpriority(-1): -1 (expected -1) ok
setpriority(50): -1 (expected -1) ok
priority after bad calls: 25 (expected 25) ok
setpriority(49): 0 (expected 0) ok
setpriority(0): 0 (expected 0) ok
getprocs(bad pointer): -1 (expected -1) ok
child inherited priority: 25 (expected 25) ok
ps (parent should show 25, child 10):
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
3       sleeping        16384   25      2       prioritytest
4       sleeping        16384   10      3       prioritytest
5       running         16384   25      3       ps
all priority tests passed
$ pexec 10 ps
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
6       sleeping        16384   10      2       pexec
7       running         16384   10      6       ps
$ pexec 7 sleep 50 &
$ pexec 3 matmul 10 &
$ ps
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
```

```
14      running         16384   0       2       ps
9       sleeping        16384   7       1       pexec
10      runnable        16384   7       9       sleep
12      sleeping        16384   3       1       pexec
13      runnable        16384   3       12      matmul
$ pexec
usage: pexec priority command [args ...]
$ pexec 5
usage: pexec priority command [args ...]
$ pexec 50 ps
pexec: priority must be a number from 0 to 49
$ pexec abc ps
pexec: priority must be a number from 0 to 49
$ pexec -1 ps
pexec: priority must be a number from 0 to 49
$ pexec 5 nosuchcmd
pexec: exec nosuchcmd failed
pexec: nosuchcmd exited with status 1
$ pexec 5 sleep
usage: sleep <ticks>
pexec: sleep exited with status 1
$ echo waiting
waiting
Time: 60 ticks
$ ps
pid     state           size    prio    ppid    name
1       sleeping        16384   0       0       init
2       sleeping        20480   0       1       sh
25      running         16384   0       2       ps
$
```

The `ps` output shows each `pexec` and its command at the requested priority (7 for `sleep`, 3 for `matmul`, 10 for `ps` itself), while `init` and `sh` stay at 0. All error cases produce a message and a non-zero status.

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

How readytime is maintained. `uint readytime` was added to `struct proc` and `struct pstat`. A process becomes `RUNNABLE` from another state in exactly five places in `kernel/proc.c`, and each one now records `p->readytime = ticks` while holding `p->lock`:

Function (line)	Transition
`userinit()` (221)	first process: USED → RUNNABLE
`kfork()` (262)	new child: USED → RUNNABLE
`yield()` (660)	preempted by the timer, or `setpriority()`: RUNNING → RUNNABLE
`wakeup()` (737)	SLEEPING → RUNNABLE
`kkill()` (763)	killed while sleeping: SLEEPING → RUNNABLE

`ticks` is read without `tickslock`. `wakeup()` is called from `clockintr()` while it already holds `tickslock`, so taking the lock there would deadlock. An aligned 32-bit read is atomic on RISC-V, so the value is never torn.

How age is calculated. `ps` calls `getprocs()` and then `uptime()` to get the current tick count. For a process whose state is `RUNNABLE` it prints `age = now - readytime`, the number of ticks it has been waiting in the ready queue. Running, sleeping and zombie processes are not waiting for the CPU, so their age is printed as 0. `uptime()` is called after `getprocs()`, so `now ≥ readytime` and the unsigned subtraction cannot wrap.

Results (still the round-robin scheduler at this commit; three CPU-bound `matmul`s run together):

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
3       running         16384   0       0       2       ps
$ sleep 100 &
$ matmul 10 &; matmul 10 &; matmul 10 &
$ ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
12      running         16384   0       0       2       ps
5       runnable        16384   2       0       1       sleep
11      runnable        16384   2       0       1       matmul
8       runnable        16384   1       0       1       matmul
10      runnable        16384   0       0       1       matmul
$ ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
13      running         16384   0       0       2       ps
5       runnable        16384   2       0       1       sleep
11      runnable        16384   2       0       1       matmul
8       runnable        16384   1       0       1       matmul
10      runnable        16384   0       0       1       matmul
$ ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
14      running         16384   0       0       2       ps
5       runnable        16384   2       0       1       sleep
11      runnable        16384   2       0       1       matmul
8       runnable        16384   1       0       1       matmul
10      runnable        16384   0       0       1       matmul
$ echo waiting
waiting
$ Time: 174 ticks
Time: 175 ticks
Time: 175 ticks
ps
pid     state           size    age     prio    ppid    name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   0       0       1       sh
16      running         16384   0       0       2       ps
$
```

Under round robin a runnable process waits at most one turn of the other runnable processes, so with four of them the ages stay between 0 and 2 ticks. Each of the three `matmul`s took about 3× its solo time (175 vs. 60 ticks), the expected even split of one CPU.

Task 4. (15 pts) Implement a priority-based scheduler.

Files and functions changed.

File / function	Change	Purpose
`kernel/param.h` (lines 17 –23)	`SCHED_RR 0`, `SCHED_PRIO 1`, and `SCHEDPOLICY` (default `SCHED_PRIO`), wrapped in `#ifndef`.	Selects the scheduler at compile time, as the handout asks.
`Makefile`	`ifdef SCHEDPOLICY` → `CFLAGS += -DSCHEDPOLICY=...`; `_schedtest` added.	Lets me build the other policy for comparison without editing `param.h`. Requires `make clean` first (see difficulties).
`kernel/proc.c` `scheduler()` (line 550)	`#if SCHEDPOLICY == SCHED_PRIO` priority loop; the original round-robin loop is kept under `#else`.	Chooses the runnable process with the highest priority.
`kernel/sysproc.c` `sys_setpriority()`	Calls `yield()` after changing the priority.	If a process lowers its own priority, a higher-priority process runs immediately instead of at the next tick.
`kernel/pstat.h`, `kgetprocs()`, `user/ps.c`	`cputime` (from PA1) added to `struct pstat` and to `ps`.	Shows which process is actually getting the CPU, like the handout's sample.
`user/schedtest.c` (new)	Test program, described below.	Measures response and turnaround time.

How the priority scheduler works. Each pass through `scheduler()` scans the whole process table. While holding each process's lock, it remembers the `RUNNABLE` process with the highest priority. Ties go to the earliest `readytime`, the process that has waited longest, so processes of equal priority take turns exactly as under round robin. It then re-locks the chosen process, re-checks that it is still `RUNNABLE` (on a multi-CPU build another CPU could have taken it between the scan and the lock), and switches to it. Because the timer interrupt calls `yield()` every tick, the choice is made again at least once per tick, so a newly runnable higher-priority process waits at most one tick.

```
#if SCHEDPOLICY == SCHED_PRIO
    struct proc *best = 0;

    for (p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      // Among equal priorities, pick the one that has waited longest,
      // so they take turns as under round robin.
      if (p->state == RUNNABLE &&
          (best == 0 || p->priority > best->priority ||
           (p->priority == best->priority &&
            p->readytime < best->readytime)))
        best = p;
      release(&p->lock);
    }

    if (best) {
      p = best;
      found = 1;
      acquire(&p->lock);
      // Another CPU may have taken it since the scan above.
      if (p->state == RUNNABLE) {
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        mycpu()->intena = 0;
        c->proc = 0;
      }
      release(&p->lock);
    }
#else
```

schedtest. `schedtest work p1 p2 …` forks one CPU-bound child per priority. Each child sets its priority and blocks reading a pipe. When all have blocked, the parent records `t0` and writes one integer per child to the pipe. That single `write()` makes every child runnable in the same tick, so they all compete from the same starting point. Each child records `uptime()` as soon as it gets the CPU (response time = first run − t0), spins for `work` units of about one tick each, and exits with its response time as its status. The parent runs at priority 49 so it notices each exit at once, gets the child's CPU time with PA1's `wait2()`, and prints response, turnaround (finish − t0) and cputime per child, plus the averages. It validates its arguments and handles pipe, fork, write and wait2 failures.

Results with the priority scheduler. Three `schedtest` runs, then a sub-shell started with `pexec 20 sh` (explained under difficulties) that runs a priority-0 and a priority-5 `matmul`, and finally a priority-5 job started from the normal priority-0 shell:

```
xv6 kernel is booting


init: starting sh
$ schedtest 20 0 10 20 30
finish  pid     prio    response        turnaround      cputime
1       7       30      0               18              18
2       6       20      18              37              19
3       5       10      37              55              18
4       4       0       55              74              19
average response: 27.5 ticks, average turnaround: 46.0 ticks
$ schedtest 20 30 20 10 0
finish  pid     prio    response        turnaround      cputime
1       9       30      0               18              18
2       10      20      18              37              19
3       11      10      37              55              18
```

```
4       12      0       55              74              19
average response: 27.5 ticks, average turnaround: 46.0 ticks
$ schedtest 20 5 5 5
finish  pid     prio    response        turnaround      cputime
1       14      5       0               54              18
2       16      5       2               55              18
3       15      5       1               55              19
average response: 1.0 ticks, average turnaround: 54.6 ticks
$ pexec 20 sh
$ pexec 0 matmul 10 &; pexec 5 matmul 10 &
$ ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
17      sleeping        16384   0       20      0       2       pexec
18      sleeping        20480   0       20      0       17      sh
23      runnable        16384   0       5       15      22      matmul
22      sleeping        16384   0       5       0       1       pexec
21      sleeping        16384   0       0       0       1       pexec
24      runnable        16384   15      0       0       21      pexec
25      running         16384   0       20      0       18      ps
$ sleep 20
$ ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
17      sleeping        16384   0       20      0       2       pexec
18      sleeping        20480   0       20      0       17      sh
23      runnable        16384   0       5       48      22      matmul
22      sleeping        16384   0       5       0       1       pexec
21      sleeping        16384   0       0       0       1       pexec
24      runnable        16384   48      0       0       21      pexec
27      running         16384   0       20      0       18      ps
$ echo waiting
waiting
$ Time: 60 ticks
ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
17      sleeping        16384   0       20      0       2       pexec
18      sleeping        20480   0       20      0       17      sh
29      running         16384   0       20      0       18      ps
21      sleeping        16384   0       0       0       1       pexec
24      runnable        16384   0       0       4       21      matmul
$ echo waiting
waiting
$ Time: 59 ticks
^D$ pexec 5 matmul 10 &
$ ps
Time: 60 ticks
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
34      running         16384   0       0       0       2       ps
$ echo waiting
waiting
$
```

Higher priority runs first. Whether the priorities were created in increasing (`0 10 20 30`) or decreasing order, the children finish strictly by priority: 30 first, after 18 ticks, and 0 last. Each lower-priority child's response time equals the run time of everything above it (0, 18, 37, 55). Equal priorities (`5 5 5`) share the CPU round-robin. In the `matmul` test the priority-0 job had 0 ticks of CPU after 48 ticks and had not even reached `exec()` (`ps` still shows its name as `pexec`)

until the priority-5 job finished. In the last test, `ps` typed at the priority-0 shell did not run until the priority-5 `matmul` printed `Time: 60 ticks`. Strict priority scheduling starves the shell too.

Same tests with round robin (`make SCHEDPOLICY=SCHED_RR`) for comparison:

```
xv6 kernel is booting


init: starting sh
$ schedtest 20 0 10 20 30
finish  pid     prio    response        turnaround      cputime
1       4       0       0               73              18
2       6       20      2               73              18
3       7       30      3               73              18
4       5       10      1               74              20
average response: 1.5 ticks, average turnaround: 73.2 ticks
$ schedtest 20 30 20 10 0
finish  pid     prio    response        turnaround      cputime
1       9       30      0               74              18
2       11      10      2               74              18
3       10      20      1               74              19
4       12      0       3               74              19
average response: 1.5 ticks, average turnaround: 74.0 ticks
$ schedtest 20 5 5 5
finish  pid     prio    response        turnaround      cputime
1       14      5       0               59              19
2       15      5       1               59              20
3       16      5       2               59              20
average response: 1.0 ticks, average turnaround: 59.0 ticks
$ pexec 20 sh
$ pexec 0 matmul 10 &; pexec 5 matmul 10 &
$ ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
17      sleeping        16384   0       20      0       2       pexec
18      sleeping        20480   0       20      0       17      sh
24      runnable        16384   1       5       10      22      matmul
22      sleeping        16384   0       5       0       1       pexec
21      sleeping        16384   0       0       0       1       pexec
23      runnable        16384   0       0       11      21      matmul
25      running         16384   0       20      0       18      ps
$ sleep 20
$ ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
17      sleeping        16384   0       20      0       2       pexec
18      sleeping        20480   0       20      0       17      sh
24      runnable        16384   1       5       31      22      matmul
22      sleeping        16384   0       5       0       1       pexec
21      sleeping        16384   0       0       0       1       pexec
23      runnable        16384   0       0       32      21      matmul
27      running         16384   0       20      0       18      ps
$ echo waiting
waiting
$ Time: 112 ticks
Time: 112 ticks
ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
17      sleeping        16384   0       20      0       2       pexec
18      sleeping        20480   0       20      0       17      sh
29      running         16384   0       20      1       18      ps
$ echo waiting
```

```
waiting
$ ^D$ pexec 5 matmul 10 &
$ ps
pid     state           size    age     prio    cputime ppid    name
1       sleeping        16384   0       0       0       0       init
2       sleeping        20480   0       0       0       1       sh
33      runnable        16384   0       5       7       32      matmul
32      sleeping        16384   0       5       0       1       pexec
34      running         16384   0       0       0       2       ps
$ Time: 66 ticks
echo waiting
waiting
$
```

Under round robin the priority field is ignored: every child starts within 3 ticks and they all finish together, after about 4 × 18 ticks. The two `matmul`s finish together, after 112 ticks. (The second `echo waiting` has no `Time:` line to wait for, because both had already printed, so my test script waited until its timeout before typing the next command.)

schedtest 20 0 10 20 30	Finish order	Response (prio 30 / 20 / 10 / 0)	Avg. response	Avg. turnaround
Round robin	any (all ≈ 74)	3 / 2 / 1 / 0	1.5	73.2
Priority	30, 20, 10, 0	0 / 18 / 37 / 55	27.5	46.0

What I learned. [DRAFT written by Claude from this session's real difficulties. Rewrite it in your own words, then delete this note.]

- A strict priority scheduler is only a few lines different from round robin: the loop has to look at every runnable process before choosing, instead of running the first one it finds.
- Tie-breaking matters. Picking the lowest table slot among equal priorities would let one process monopolize its level; using `readytime` turns each priority level into a round-robin queue.
- Starvation is real and easy to cause. A single CPU-bound job at priority 5 stopped every priority-0 process, including the shell. Typed characters still echoed, because the console interrupt handler does that, but no command ran.
- With equal-length jobs, running them one after another (priority order) gives a much lower average turnaround than round robin (46 vs. 73 ticks), because round robin makes every job finish near the end.

Difficulties. [DRAFT written by Claude from this session's real difficulties. Rewrite it in your own words, then delete this note.]

- **Getting ps to run during the test.** With priority-only scheduling, the priority-0 shell could not run `ps` while a higher-priority job ran. I started a second shell at priority 20 with `pexec 20 sh` and launched the jobs from it with explicit `pexec 0` / `pexec 5`, so `ps` (priority 20) could interrupt them.
- **Measuring response time fairly.** If the parent forks children one at a time, the first child gets a head start. The pipe barrier in `schedtest` makes them all runnable in the same tick. The children also must lower their priority before blocking, or they would run at the parent's priority 49.

- **Calibrating the work unit.** My first guess, 1.5 million loop iterations per unit, was about 30× too small: `schedtest 10 0` finished in 0 ticks. I timed 200 units (7 ticks) and set `UNIT` to 40 million, about one tick in QEMU on this machine.
- **Switching policies.** `make SCHEDPOLICY=SCHED_RR` alone did not rebuild anything, because Make does not track changes to `CFLAGS`. I always run `make clean` before switching.
- **ps.c did not compile** (`%l`, see Task 1).

Task 5. (15 pts) Add aging to your priority-based scheduler.

Aging policy. A runnable process's effective priority grows by one level for every `AGINGTICKS` ticks it has been waiting since it last became runnable, up to `MAXEFFPRIORITY`:

```
effective = min(MAXEFFPRIORITY, priority + (ticks - readytime) / AGINGTICKS)
MAXEFFPRIORITY = 99, AGINGTICKS = 1      (kernel/param.h lines 25-31)
AGING = 1 by default; AGING=0 turns aging off
```

- The scheduler compares effective priorities instead of base priorities. Ties still go to the earliest `readytime`.
- As the handout suggests, there is no age field. The age is computed from `readytime` when it is needed, in `effpriority()` (`kernel/proc.c` line 489), which both `scheduler()` and `getprocs()` call. `ps` shows it in a new `eprio` column.
- Only `RUNNABLE` processes age. When a process is preempted, `yield()` resets its `readytime`, so it starts again at its base priority. Aging therefore gives a starving process one time slice and then it waits again. It never permanently raises a process's priority.
- **Guarantee:** a runnable process whose priority is d levels below the highest runnable process waits at most about d × `AGINGTICKS` ticks before it runs. Starvation is impossible, but the high-priority process still gets about d/(d+1) of the CPU.
- I chose `AGINGTICKS = 1`, the handout's example, because it keeps the guarantee easy to state and keeps priorities meaningful: a gap of 5 levels gives a 5:1 CPU share.

```
// Priority the scheduler uses for p: its base priority plus
// aging credit for the time it has been waiting to run.
// Caller must hold p->lock.
static int
effpriority(struct proc *p, uint now)
{
  uint age;


  if (!AGING || p->state != RUNNABLE)
    return p->priority;
  age = (now - p->readytime) / AGINGTICKS;
  if (age >= MAXEFFPRIORITY - p->priority)
    return MAXEFFPRIORITY;
  return p->priority + age;
}

#if SCHEDPOLICY == SCHED_PRIO
    struct proc *best = 0;
    int prio, bestprio = 0;
    uint bestready = 0;
    uint now = ticks;


    for (p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
```

```
      if (p->state == RUNNABLE) {
        prio = effpriority(p, now);
        // Among equal priorities, pick the one that has waited longest,
        // so they take turns as under round robin.
        if (best == 0 || prio > bestprio ||
            (prio == bestprio && p->readytime < bestready)) {
          best = p;
          bestprio = prio;
          bestready = p->readytime;
        }
      }
      release(&p->lock);
    }

    if (best) {
      p = best;
      found = 1;
      acquire(&p->lock);
      // Another CPU may have taken it since the scan above.
      if (p->state == RUNNABLE) {
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        mycpu()->intena = 0;
        c->proc = 0;
      }
      release(&p->lock);
    }
#else
```

Results with aging (the handout's sample scenario, typed at the normal priority-0 shell):

```
xv6 kernel is booting


init: starting sh
$ pexec 5 matmul 10 &; matmul 10 &
$ pexec 10 ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
7       runnable        16384   0       5       5       20      5       matmul
6       runnable        16384   5       0       5       4       1       matmul
5       sleeping        16384   0       5       5       0       1       pexec
8       sleeping        16384   0       10      10      0       2       pexec
9       running         16384   0       10      10      0       8       ps
$ pexec 10 ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
7       runnable        16384   0       5       5       36      5       matmul
6       runnable        16384   1       0       1       8       1       matmul
5       sleeping        16384   0       5       5       0       1       pexec
10      sleeping        16384   0       10      10      0       2       pexec
11      running         16384   0       10      10      0       10      ps
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
7       runnable        16384   0       5       5       52      5       matmul
6       runnable        16384   2       0       2       11      1       matmul
5       sleeping        16384   0       5       5       0       1       pexec
12      running         16384   0       0       0       0       2       ps
$ echo waiting
Time: 70 ticks
waiting
$ Time: 122 ticks
```

```
ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
14      running         16384   0       0       0       0       2       ps
$
```

The priority-0 `matmul` (pid 6) keeps getting CPU time while the priority-5 one runs: cputime 4 → 8 → 11 ticks versus 20 → 36 → 52. Its `eprio` column shows it climbing toward 5 before it gets a tick. The priority-0 shell also runs `ps` right away. The priority-5 job finished in 70 ticks instead of about 60, the cost of sharing.

Same commands without aging (`make AGING=0`):

```
xv6 kernel is booting


init: starting sh
$ pexec 5 matmul 10 &; matmul 10 &
$ pexec 10 ps
Time: 58 ticks
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
8       sleeping        16384   0       10      10      0       2       pexec
6       runnable        16384   0       0       0       6       1       matmul
9       running         16384   0       10      10      0       8       ps
$ pexec 10 ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
10      sleeping        16384   0       10      10      0       2       pexec
6       runnable        16384   0       0       0       10      1       matmul
11      running         16384   0       10      10      0       10      ps
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
12      running         16384   0       0       0       0       2       ps
6       runnable        16384   0       0       0       14      1       matmul
$ echo waiting
waiting
$ Time: 119 ticks
ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       0       1       sh
14      running         16384   0       0       0       0       2       ps
$
```

Without aging, the shell cannot even start `pexec 10 ps` until the priority-5 `matmul` finishes (`Time: 58 ticks` is printed before the `ps` output). By the time any `ps` runs, the priority-5 job is gone, and only then does the priority-0 `matmul` make progress (cputime 6 → 10 → 14 in the three snapshots). It finished after 119 ticks.

Evaluation: average response and turnaround time. Each `schedtest` workload was run on all three builds. Every child needs about 18 ticks of CPU. Values are averages in ticks, response / turnaround:

Workload (priorities)	Round robin	Priority, no aging	Priority + aging
`0 10 20 30`	1.5 / 78.5	27.5 / 46.2	12.5 / 47.5

Workload (priorities)	Round robin	Priority, no aging	Priority + aging
`0 5`	0.5 / 37.0	9.0 / 27.5	2.5 / 29.5
`0 45`	0.5 / 37.0	9.0 / 27.5	9.0 / 27.5
`5 5 5`	1.0 / 55.0	1.0 / 54.6	1.0 / 54.6
worst response, `0 10 20 30` (prio-0 job)	0	55	21

- **Response time:** aging cuts the average response time of the priority scheduler by more than half (27.5 → 12.5 ticks; 2.5 vs. 9.0 for a small gap). The lowest-priority job's response drops from 55 to 21 ticks: it waits roughly its 30-level gap, not for every other job to finish. Round robin still has the best response time, since it ignores priorities.
- **Turnaround time:** with equal-length jobs, running them one after another gives the lowest average turnaround, and round robin the highest. Aging costs very little turnaround (46.2 → 47.5 and 27.5 → 29.5), because it only adds a few context switches; total work is the same.
- **When aging does nothing:** with a 45-level gap and jobs only about 18 ticks long, the high-priority job finishes before the low one has aged enough, so the results equal no-aging. With equal priorities all three schedulers behave the same.
- Run-to-run variation is about ±1 tick on short runs (for example, the round-robin `0 10 20 30` average turnaround was 73.2 in Task 4 and 78.5 here) because QEMU's speed varies with host load.

Evaluation transcripts (priority + aging, priority without aging, round robin):

Priority + aging (default build):

```
xv6 kernel is booting


init: starting sh
$ schedtest 20 0 10 20 30
finish  pid     prio    response        turnaround      cputime
1       7       30      0               19              18
2       6       20      10              39              18
3       5       10      19              58              19
4       4       0       21              74              19
average response: 12.5 ticks, average turnaround: 47.5 ticks
$ schedtest 20 0 5
finish  pid     prio    response        turnaround      cputime
1       10      5       0               21              18
2       9       0       5               38              20
average response: 2.5 ticks, average turnaround: 29.5 ticks
$ schedtest 20 0 45
finish  pid     prio    response        turnaround      cputime
1       13      45      0               18              18
2       12      0       18              37              19
average response: 9.0 ticks, average turnaround: 27.5 ticks
$ schedtest 20 5 5 5
finish  pid     prio    response        turnaround      cputime
1       15      5       0               54              18
2       17      5       2               55              18
3       16      5       1               55              19
average response: 1.0 ticks, average turnaround: 54.6 ticks
$
```

Priority without aging (`make AGING=0`):

```
xv6 kernel is booting

init: starting sh
$ schedtest 20 0 10 20 30
finish  pid     prio    response        turnaround      cputime
1       7       30      0               18              18
2       6       20      18              37              19
3       5       10      37              55              18
4       4       0       55              75              20
average response: 27.5 ticks, average turnaround: 46.2 ticks
$ schedtest 20 0 5
finish  pid     prio    response        turnaround      cputime
1       10      5       0               18              18
2       9       0       18              37              19
average response: 9.0 ticks, average turnaround: 27.5 ticks
$ schedtest 20 0 45
finish  pid     prio    response        turnaround      cputime
1       13      45      0               18              18
2       12      0       18              37              19
average response: 9.0 ticks, average turnaround: 27.5 ticks
$ schedtest 20 5 5 5
finish  pid     prio    response        turnaround      cputime
1       15      5       0               54              18
2       17      5       2               55              18
3       16      5       1               55              19
average response: 1.0 ticks, average turnaround: 54.6 ticks
$
```

Round robin (`make SCHEDPOLICY=SCHED_RR`):

```
xv6 kernel is booting

init: starting sh
$ schedtest 20 0 10 20 30
finish  pid     prio    response        turnaround      cputime
1       5       10      1               78              19
2       7       30      3               78              19
3       4       0       0               79              20
4       6       20      2               79              21
average response: 1.5 ticks, average turnaround: 78.5 ticks
$ schedtest 20 0 5
finish  pid     prio    response        turnaround      cputime
1       9       0       0               37              18
2       10      5       1               37              19
average response: 0.5 ticks, average turnaround: 37.0 ticks
$ schedtest 20 0 45
finish  pid     prio    response        turnaround      cputime
1       12      0       0               37              18
2       13      45      1               37              19
average response: 0.5 ticks, average turnaround: 37.0 ticks
$ schedtest 20 5 5 5
finish  pid     prio    response        turnaround      cputime
1       15      5       0               55              18
2       17      5       2               55              18
3       16      5       1               55              19
average response: 1.0 ticks, average turnaround: 55.0 ticks
$
```

What I learned. [DRAFT written by Claude from this session's real difficulties. Rewrite it in your own words, then delete this note.]

- Aging turns "never" into "after about d ticks" for a process d levels below, and the priority still controls the CPU share.

- Resetting `readytime` on every transition to RUNNABLE matters: a process loses its aging credit after each time slice, so aging cannot let a low-priority process take over.
- Response time and turnaround time pull in opposite directions. Aging moves the priority scheduler toward round robin's good response at very little turnaround cost.
- The effect depends on the size of the priority gap relative to job length, so the aging rate (`AGINGTICKS`) is a real tuning knob.

Difficulties. [DRAFT written by Claude from this session's real difficulties. Rewrite it in your own words, then delete this note.]

- **Locking for the effective priority.** The scheduler compares the current process against the best one found so far, but it no longer holds the best one's lock. I store the best effective priority and `readytime` in local variables instead of re-reading the other process's fields without its lock.
- **Overflow in the formula.** `priority + age` with an unsigned age could overflow for very old processes, so `effpriority()` checks `age >= MAXEFFPRIORITY - priority` before adding.
- **Making the benefit visible.** In the `matmul` test aging mostly shows up as a responsive shell. The `schedtest` table with different priority gaps (5, 30, 45) shows where aging helps and where it cannot.

Task 6. (5 pts) Summary

[DRAFT written by Claude from this session's real difficulties. Rewrite it in your own words, then delete this note.]

This assignment showed me how small the scheduler's core decision is, and how much depends on it. The xv6 scheduler is one loop. Changing what that loop picks, from "next runnable" to "highest priority", changed how the whole system behaved, down to whether the shell could run a command. Adding `getprocs()` and `ps` first was useful because it let me see the scheduler's decisions (state, age, priority, CPU time) instead of guessing from timing output.

The main obstacles were:

- The provided `ps.c` did not compile on the current xv6 (`%l` → `%ld`).
- `enum procstate` lived in a kernel-only header, so I moved it to `pstat.h`.
- Under strict priority scheduling the shell was starved, so I tested from a high-priority sub-shell.
- I needed a fair way to measure response time, which led to the pipe barrier in `schedtest`, and I had to calibrate its work unit (first guess 30× too small).
- Switching scheduler builds needs `make clean`.
- Setup: `gh` was configured for SSH, which is not set up on my machine, so the repository is cloned over HTTPS.

All of xv6's own tests still pass with the new scheduler (`usertests -q`: ALL TESTS PASSED, below).

Extra Credit Task (10 points).

Implementation. Like `cd`, these commands must run inside the shell process itself. If `sh` forked a child to run `setpriority`, the child would change its own priority and then exit, and the shell would be unchanged. In `user/sh.c`, `main()` (line 222) now calls `prioritycmd()` (line 153) before forking. `prioritycmd()` splits a copy of the line into words. If the first word is `getpriority` it prints `getpriority()`. If it is `setpriority N`, it checks that N is a number and calls the `setpriority()` system call on the shell's own process. Wrong argument counts, non-numbers and out-of-range values print a usage or error message. Any other command returns 0 and runs normally. Because `fork()` copies the priority, every command started afterwards runs at the shell's new priority.

```
// Run the getpriority and setpriority builtins. Like cd, they must
// run in the shell itself: a child's priority change would be lost
// when it exits. Children of the shell inherit its priority.
// Return 1 if cmd was one of them, 0 otherwise.
int
prioritycmd(char *cmd)
{
  char line[100], *argv[3], *s;
  int argc;

  strcpy(line, cmd);
  argc = 0;
  for (s = line; *s;) {
    while (*s && strchr(whitespace, *s))
      s++;
    if (*s == 0)
      break;
    if (argc < 3)
      argv[argc] = s;
    argc++;
    while (*s && !strchr(whitespace, *s))
      s++;
    if (*s)
      *s++ = 0;
  }

  if (argc > 0 && strcmp(argv[0], "getpriority") == 0) {
    if (argc != 1)
      fprintf(2, "usage: getpriority\n");
    else
      printf("%d\n", getpriority());
    return 1;
  }
  if (argc > 0 && strcmp(argv[0], "setpriority") == 0) {
    if (argc != 2) {
      fprintf(2, "usage: setpriority priority\n");
      return 1;
    }
    for (s = argv[1]; *s >= '0' && *s <= '9'; s++)
      ;
    if (s == argv[1] || *s != 0 || setpriority(atoi(argv[1])) < 0)
      fprintf(2, "setpriority: priority must be a number from 0 to %d\n",
              MAXPRIORITY);
    return 1;
  }
  return 0;
}
```

Results. The handout's sample, error cases, and then `time wc README` (an interactive job that reads the disk and prints) with two CPU-bound `matmul`s at priority 20 in the background:

```
xv6 kernel is booting


init: starting sh
$ getpriority
0
$ setpriority 10
$ getpriority
10
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       10      10      0       1       sh
3       running         16384   0       10      10      0       2       ps
$ setpriority
usage: setpriority priority
$ setpriority 50
setpriority: priority must be a number from 0 to 49
$ setpriority abc
setpriority: priority must be a number from 0 to 49
$ setpriority 5 6
usage: setpriority priority
$ getpriority now
usage: getpriority
$ getpriority
10
$ setpriority 0
$ time wc README
48 336 2441 README
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ pexec 20 matmul 20 &; pexec 20 matmul 20 &
$ time wc README
48 336 2441 README
elapsed time: 42 ticks, cpu time: 0 ticks, 0% CPU
time wc README
$ 48 336 2441 README
elapsed time: 42 ticks, cpu time: 0 ticks, 0% CPU
setpriority 30
$ $ time wc README
48 336 2441 README
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ time wc README
48 336 2441 README
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       30      30      0       1       sh
10      runnable        16384   0       20      20      106     9       matmul
9       sleeping        16384   0       20      20      0       1       pexec
8       sleeping        16384   0       20      20      0       1       pexec
11      runnable        16384   1       20      21      105     8       matmul
20      running         16384   0       30      30      0       2       ps
$ echo waiting
waiting
$ Time: 256 ticks
Time: 255 ticks
```

`time wc README`	**Shell priority**	**Elapsed ticks**
No background load	0	0
Two `matmul`s at priority 20 running	0	42, 42

`time wc README`	Shell priority	Elapsed ticks
Same load, after `setpriority 30`	30	0, 0

With the shell at priority 0, `wc` waits behind the priority-20 jobs every time it wakes from a disk read, until aging lifts it 20 levels: 42 ticks (4 seconds) for a command that normally takes no time. After `setpriority 30` the shell and everything it starts outrank the batch jobs, so the response is immediate again. The background jobs still share the CPU evenly (`ps` shows 106 and 105 ticks) and finished in 255–256 ticks, about twice the time of one `matmul 20` alone, because an interactive job uses very little CPU. Giving interactive work a high priority and batch work a low one is how real schedulers keep a loaded system responsive.

Regression test: usertests

`usertests -q` on the final kernel (default build: priority scheduler with aging, `CPUS := 1`). It ends with ALL TESTS PASSED. The many `usertrap(): unexpected scause` lines are expected: tests such as `kernmem`, `MAXVAplus` and `lazy_unmap` deliberately make child processes touch invalid memory, and the kernel kills each one, which is what those tests check. The full transcript, verbatim:

```
xv6 kernel is booting


init: starting sh
$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test openiput: OK
test exitiput: OK
test iput: OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test dirtest: OK
test exectest: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test reparent: OK
test twochildren: OK
test forkfork: OK
test forkforkfork: OK
test reparent2: OK
test mem: OK
test sharedfd: OK
test fourfiles: OK
test createdelete: OK
test unlinkread: OK
test linktest: OK
test concreate: OK
test linkunlink: OK
test subdir: OK
test bigwrite: OK
test bigfile: OK
test fourteen: OK
test rmdot: OK
test dirfile: OK
```

```
test iref: OK
test forktest: OK
test sbrkbasic: OK
test sbrkmuch: OK
test kernmem: usertrap(): unexpected scause 0xd pid=6020
            sepc=0x1b5a stval=0x80000000
usertrap(): unexpected scause 0xd pid=6021
            sepc=0x1b5a stval=0x8000c350
usertrap(): unexpected scause 0xd pid=6022
            sepc=0x1b5a stval=0x800186a0
usertrap(): unexpected scause 0xd pid=6023
            sepc=0x1b5a stval=0x800249f0
usertrap(): unexpected scause 0xd pid=6024
            sepc=0x1b5a stval=0x80030d40
usertrap(): unexpected scause 0xd pid=6025
            sepc=0x1b5a stval=0x8003d090
usertrap(): unexpected scause 0xd pid=6026
            sepc=0x1b5a stval=0x800493e0
usertrap(): unexpected scause 0xd pid=6027
            sepc=0x1b5a stval=0x80055730
usertrap(): unexpected scause 0xd pid=6028
            sepc=0x1b5a stval=0x80061a80
usertrap(): unexpected scause 0xd pid=6029
            sepc=0x1b5a stval=0x8006ddd0
usertrap(): unexpected scause 0xd pid=6030
            sepc=0x1b5a stval=0x8007a120
usertrap(): unexpected scause 0xd pid=6031
            sepc=0x1b5a stval=0x80086470
usertrap(): unexpected scause 0xd pid=6032
            sepc=0x1b5a stval=0x800927c0
usertrap(): unexpected scause 0xd pid=6033
            sepc=0x1b5a stval=0x8009eb10
usertrap(): unexpected scause 0xd pid=6034
            sepc=0x1b5a stval=0x800aae60
usertrap(): unexpected scause 0xd pid=6035
            sepc=0x1b5a stval=0x800b71b0
usertrap(): unexpected scause 0xd pid=6036
            sepc=0x1b5a stval=0x800c3500
usertrap(): unexpected scause 0xd pid=6037
            sepc=0x1b5a stval=0x800cf850
usertrap(): unexpected scause 0xd pid=6038
            sepc=0x1b5a stval=0x800dbba0
usertrap(): unexpected scause 0xd pid=6039
            sepc=0x1b5a stval=0x800e7ef0
usertrap(): unexpected scause 0xd pid=6040
            sepc=0x1b5a stval=0x800f4240
usertrap(): unexpected scause 0xd pid=6041
            sepc=0x1b5a stval=0x80100590
usertrap(): unexpected scause 0xd pid=6042
            sepc=0x1b5a stval=0x8010c8e0
usertrap(): unexpected scause 0xd pid=6043
            sepc=0x1b5a stval=0x80118c30
usertrap(): unexpected scause 0xd pid=6044
            sepc=0x1b5a stval=0x80124f80
usertrap(): unexpected scause 0xd pid=6045
            sepc=0x1b5a stval=0x801312d0
usertrap(): unexpected scause 0xd pid=6046
            sepc=0x1b5a stval=0x8013d620
usertrap(): unexpected scause 0xd pid=6047
            sepc=0x1b5a stval=0x80149970
usertrap(): unexpected scause 0xd pid=6048
            sepc=0x1b5a stval=0x80155cc0
usertrap(): unexpected scause 0xd pid=6049
            sepc=0x1b5a stval=0x80162010
usertrap(): unexpected scause 0xd pid=6050
            sepc=0x1b5a stval=0x8016e360
usertrap(): unexpected scause 0xd pid=6051
            sepc=0x1b5a stval=0x8017a6b0
usertrap(): unexpected scause 0xd pid=6052
            sepc=0x1b5a stval=0x80186a00
usertrap(): unexpected scause 0xd pid=6053
            sepc=0x1b5a stval=0x80192d50
usertrap(): unexpected scause 0xd pid=6054
            sepc=0x1b5a stval=0x8019f0a0
usertrap(): unexpected scause 0xd pid=6055
            sepc=0x1b5a stval=0x801ab3f0
usertrap(): unexpected scause 0xd pid=6056
            sepc=0x1b5a stval=0x801b7740
usertrap(): unexpected scause 0xd pid=6057
            sepc=0x1b5a stval=0x801c3a90
usertrap(): unexpected scause 0xd pid=6058
```

```
            sepc=0x1b5a stval=0x801cfde0
usertrap(): unexpected scause 0xd pid=6059
            sepc=0x1b5a stval=0x801dc130
OK
test MAXVAplus: usertrap(): unexpected scause 0xf pid=6061
            sepc=0x1bf0 stval=0x4000000000
usertrap(): unexpected scause 0xf pid=6062
            sepc=0x1bf0 stval=0x8000000000
usertrap(): unexpected scause 0xf pid=6063
            sepc=0x1bf0 stval=0x10000000000
usertrap(): unexpected scause 0xf pid=6064
            sepc=0x1bf0 stval=0x20000000000
usertrap(): unexpected scause 0xf pid=6065
            sepc=0x1bf0 stval=0x40000000000
usertrap(): unexpected scause 0xf pid=6066
            sepc=0x1bf0 stval=0x80000000000
usertrap(): unexpected scause 0xf pid=6067
            sepc=0x1bf0 stval=0x100000000000
usertrap(): unexpected scause 0xf pid=6068
            sepc=0x1bf0 stval=0x200000000000
usertrap(): unexpected scause 0xf pid=6069
            sepc=0x1bf0 stval=0x400000000000
usertrap(): unexpected scause 0xf pid=6070
            sepc=0x1bf0 stval=0x800000000000
usertrap(): unexpected scause 0xf pid=6071
            sepc=0x1bf0 stval=0x1000000000000
usertrap(): unexpected scause 0xf pid=6072
            sepc=0x1bf0 stval=0x2000000000000
usertrap(): unexpected scause 0xf pid=6073
            sepc=0x1bf0 stval=0x4000000000000
usertrap(): unexpected scause 0xf pid=6074
            sepc=0x1bf0 stval=0x8000000000000
usertrap(): unexpected scause 0xf pid=6075
            sepc=0x1bf0 stval=0x10000000000000
usertrap(): unexpected scause 0xf pid=6076
            sepc=0x1bf0 stval=0x20000000000000
usertrap(): unexpected scause 0xf pid=6077
            sepc=0x1bf0 stval=0x40000000000000
usertrap(): unexpected scause 0xf pid=6078
            sepc=0x1bf0 stval=0x80000000000000
usertrap(): unexpected scause 0xf pid=6079
            sepc=0x1bf0 stval=0x100000000000000
usertrap(): unexpected scause 0xf pid=6080
            sepc=0x1bf0 stval=0x200000000000000
usertrap(): unexpected scause 0xf pid=6081
            sepc=0x1bf0 stval=0x400000000000000
usertrap(): unexpected scause 0xf pid=6082
            sepc=0x1bf0 stval=0x800000000000000
usertrap(): unexpected scause 0xf pid=6083
            sepc=0x1bf0 stval=0x1000000000000000
usertrap(): unexpected scause 0xf pid=6084
            sepc=0x1bf0 stval=0x2000000000000000
usertrap(): unexpected scause 0xf pid=6085
            sepc=0x1bf0 stval=0x4000000000000000
usertrap(): unexpected scause 0xf pid=6086
            sepc=0x1bf0 stval=0x8000000000000000
OK
test sbrkfail: OK
test sbrkarg: OK
test validatetest: OK
test bsstest: OK
test bigargtest: OK
test argptest: OK
test stacktest: usertrap(): unexpected scause 0xd pid=6106
            sepc=0x1c44 stval=0x10e90
OK
test nowrite: usertrap(): unexpected scause 0xf pid=6108
            sepc=0x1cec stval=0x0
usertrap(): unexpected scause 0xf pid=6109
            sepc=0x1cec stval=0x80000000
usertrap(): unexpected scause 0xf pid=6110
            sepc=0x1cec stval=0x3fffffe000
usertrap(): unexpected scause 0xf pid=6111
            sepc=0x1cec stval=0x3ffffff000
usertrap(): unexpected scause 0xf pid=6112
            sepc=0x1cec stval=0x4000000000
usertrap(): unexpected scause 0xf pid=6113
            sepc=0x1cec stval=0xffffffffffffffff
OK
test pgbug: OK
test sbrkbugs: usertrap(): unexpected scause 0xc pid=6116
```

```
            sepc=0x5764 stval=0x5764
usertrap(): unexpected scause 0xc pid=6117
            sepc=0x5764 stval=0x5764
usertrap(): unexpected scause 0xd pid=6118
            sepc=0x56b0 stval=0x11ea8
OK
test sbrklast: OK
test sbrk8000: OK
test badarg: OK
test lazy_alloc: OK
test lazy_unmap: usertrap(): unexpected scause 0xf pid=6124
            sepc=0x49ba stval=0x13000
usertrap(): unexpected scause 0xf pid=6125
            sepc=0x49ba stval=0x1013000
usertrap(): unexpected scause 0xf pid=6126
            sepc=0x49ba stval=0x2013000
usertrap(): unexpected scause 0xf pid=6127
            sepc=0x49ba stval=0x3013000
usertrap(): unexpected scause 0xf pid=6128
            sepc=0x49ba stval=0x4013000
usertrap(): unexpected scause 0xf pid=6129
            sepc=0x49ba stval=0x5013000
usertrap(): unexpected scause 0xf pid=6130
            sepc=0x49ba stval=0x6013000
usertrap(): unexpected scause 0xf pid=6131
            sepc=0x49ba stval=0x7013000
usertrap(): unexpected scause 0xf pid=6132
            sepc=0x49ba stval=0x8013000
usertrap(): unexpected scause 0xf pid=6133
            sepc=0x49ba stval=0x9013000
usertrap(): unexpected scause 0xf pid=6134
            sepc=0x49ba stval=0xa013000
usertrap(): unexpected scause 0xf pid=6135
            sepc=0x49ba stval=0xb013000
usertrap(): unexpected scause 0xf pid=6136
            sepc=0x49ba stval=0xc013000
usertrap(): unexpected scause 0xf pid=6137
            sepc=0x49ba stval=0xd013000
usertrap(): unexpected scause 0xf pid=6138
            sepc=0x49ba stval=0xe013000
usertrap(): unexpected scause 0xf pid=6139
            sepc=0x49ba stval=0xf013000
usertrap(): unexpected scause 0xf pid=6140
            sepc=0x49ba stval=0x10013000
usertrap(): unexpected scause 0xf pid=6141
            sepc=0x49ba stval=0x11013000
usertrap(): unexpected scause 0xf pid=6142
            sepc=0x49ba stval=0x12013000
usertrap(): unexpected scause 0xf pid=6143
            sepc=0x49ba stval=0x13013000
usertrap(): unexpected scause 0xf pid=6144
            sepc=0x49ba stval=0x14013000
usertrap(): unexpected scause 0xf pid=6145
            sepc=0x49ba stval=0x15013000
usertrap(): unexpected scause 0xf pid=6146
            sepc=0x49ba stval=0x16013000
usertrap(): unexpected scause 0xf pid=6147
            sepc=0x49ba stval=0x17013000
usertrap(): unexpected scause 0xf pid=6148
            sepc=0x49ba stval=0x18013000
usertrap(): unexpected scause 0xf pid=6149
            sepc=0x49ba stval=0x19013000
usertrap(): unexpected scause 0xf pid=6150
            sepc=0x49ba stval=0x1a013000
usertrap(): unexpected scause 0xf pid=6151
            sepc=0x49ba stval=0x1b013000
usertrap(): unexpected scause 0xf pid=6152
            sepc=0x49ba stval=0x1c013000
usertrap(): unexpected scause 0xf pid=6153
            sepc=0x49ba stval=0x1d013000
usertrap(): unexpected scause 0xf pid=6154
            sepc=0x49ba stval=0x1e013000
usertrap(): unexpected scause 0xf pid=6155
            sepc=0x49ba stval=0x1f013000
usertrap(): unexpected scause 0xf pid=6156
            sepc=0x49ba stval=0x20013000
usertrap(): unexpected scause 0xf pid=6157
            sepc=0x49ba stval=0x21013000
usertrap(): unexpected scause 0xf pid=6158
            sepc=0x49ba stval=0x22013000
usertrap(): unexpected scause 0xf pid=6159
```

```
            sepc=0x49ba stval=0x23013000
usertrap(): unexpected scause 0xf pid=6160
            sepc=0x49ba stval=0x24013000
usertrap(): unexpected scause 0xf pid=6161
            sepc=0x49ba stval=0x25013000
usertrap(): unexpected scause 0xf pid=6162
            sepc=0x49ba stval=0x26013000
usertrap(): unexpected scause 0xf pid=6163
            sepc=0x49ba stval=0x27013000
usertrap(): unexpected scause 0xf pid=6164
            sepc=0x49ba stval=0x28013000
usertrap(): unexpected scause 0xf pid=6165
            sepc=0x49ba stval=0x29013000
usertrap(): unexpected scause 0xf pid=6166
            sepc=0x49ba stval=0x2a013000
usertrap(): unexpected scause 0xf pid=6167
            sepc=0x49ba stval=0x2b013000
usertrap(): unexpected scause 0xf pid=6168
            sepc=0x49ba stval=0x2c013000
usertrap(): unexpected scause 0xf pid=6169
            sepc=0x49ba stval=0x2d013000
usertrap(): unexpected scause 0xf pid=6170
            sepc=0x49ba stval=0x2e013000
usertrap(): unexpected scause 0xf pid=6171
            sepc=0x49ba stval=0x2f013000
usertrap(): unexpected scause 0xf pid=6172
            sepc=0x49ba stval=0x30013000
usertrap(): unexpected scause 0xf pid=6173
            sepc=0x49ba stval=0x31013000
usertrap(): unexpected scause 0xf pid=6174
            sepc=0x49ba stval=0x32013000
usertrap(): unexpected scause 0xf pid=6175
            sepc=0x49ba stval=0x33013000
usertrap(): unexpected scause 0xf pid=6176
            sepc=0x49ba stval=0x34013000
usertrap(): unexpected scause 0xf pid=6177
            sepc=0x49ba stval=0x35013000
usertrap(): unexpected scause 0xf pid=6178
            sepc=0x49ba stval=0x36013000
usertrap(): unexpected scause 0xf pid=6179
            sepc=0x49ba stval=0x37013000
usertrap(): unexpected scause 0xf pid=6180
            sepc=0x49ba stval=0x38013000
usertrap(): unexpected scause 0xf pid=6181
            sepc=0x49ba stval=0x39013000
usertrap(): unexpected scause 0xf pid=6182
            sepc=0x49ba stval=0x3a013000
usertrap(): unexpected scause 0xf pid=6183
            sepc=0x49ba stval=0x3b013000
usertrap(): unexpected scause 0xf pid=6184
            sepc=0x49ba stval=0x3c013000
usertrap(): unexpected scause 0xf pid=6185
            sepc=0x49ba stval=0x3d013000
usertrap(): unexpected scause 0xf pid=6186
            sepc=0x49ba stval=0x3e013000
usertrap(): unexpected scause 0xf pid=6187
            sepc=0x49ba stval=0x3f013000
OK
test lazy_copy: OK
test lazy_copyinstr: OK
test lazy_sbrk: OK
test partial_write: OK
test unlinkcwd: OK
ALL TESTS PASSED
$
```

Results summary

Item	Evidence	Result
Task 1 getprocs / ps	ps with idle system and background jobs; bad-pointer getprocs returns -1	Pass
Task 2 priorities	prioritytest (10 checks), pexec at 3/7/10, 7 pexec error cases	Pass

Item	Evidence	Result
Task 3 readytime / age	ps with three concurrent matmuls: ages 0–2 under round robin	Pass
Task 4 priority scheduler	schedtest finishes strictly by priority; RR build for comparison	Pass
Task 5 aging	low-priority job runs during high-priority job; response 27.5 → 12.5 ticks	Pass
Extra credit	getpriority/setpriority builtins; time wc README 42 → 0 ticks under load	Pass
Regression	usertests -q	ALL TESTS PASSED

Resources and AI-use disclosure

Resources: the assignment handout and the provided `ps.c` and `pexec.c`; the xv6-riscv source (github.com/mit-pdos/xv6-riscv); the xv6 book, chapter 7 (scheduling); and my PA1 code (`time`, `matmul`, `wait2`, `cputime`).

AI use: Claude (Anthropic's Claude Opus 5.5 model, used through Claude Code) did the following. (1) Wrote all of the code on the `pa2` branch: the `getprocs`, `getpriority` and `setpriority` system calls, the `priority`/`readytime` fields, the priority scheduler and the aging policy (it chose the tie-breaking rule and `AGINGTICKS = 1`), the `param.h`/Makefile configuration, the changes to the provided `ps.c` and `pexec.c`, the new test programs `prioritytest.c` and `schedtest.c`, and the shell builtins for the extra credit. (2) Built xv6 and ran every test in QEMU with a Python script that types the commands, including `usertests`. All output in this report is copied from those runs. (3) Wrote this report, including the drafts of the "What I learned", "Difficulties" and Task 6 sections, which are based on problems that actually came up while it worked. (4) Wrote `demo.py` and a demo walkthrough for the grading demo. I provided the handout, the template, my PA1 repository and the instructions. [REVIEW AND EDIT: describe what you checked, changed, and understood yourself, then remove this bracketed note and the red color.]